Karachi AQI Predictor

End-to-End Air Quality Forecasting System using Hopsworks, Machine Learning, and Streamlit

Alisha Ather

DataScience Intern

Abstract

Air pollution has become one of the most pressing environmental and public health challenges facing cities such as Karachi, where pollutant concentrations regularly exceed safe thresholds. This report documents the design and development of an end-to-end Air Quality Index (AQI) forecasting system for the city, capable of forecasting AQI 24, 48, and 72 hours in advance.

The system was built around modern MLOps practices rather than a single notebook workflow. An automated feature pipeline retrieves historical and live weather and pollutant data from the Open-Meteo API, engineers time-based, lag, rolling, and rate-of-change features, and writes the results to the Hopsworks Feature Store. From there, a training pipeline performs time-aware model comparison and hyperparameter tuning, and registers the best-performing model, a Gradient Boosting model, to the Hopsworks Model Registry. Forecasts are served through a Streamlit dashboard that includes SHAP-based explainability and hazard alerts. Both pipelines run automatically through GitHub Actions, with the feature pipeline scheduled hourly and the training pipeline scheduled daily.

The final tuned model outperformed a naive persistence baseline at every forecast horizon. RMSE and MAE were used as the primary metrics for model selection, rather than R^2, since R^2 proved to be a poor indicator of performance on a test window with relatively low variance, a limitation this report investigates.

1. Introduction

The Air Quality Index is a standardized way of expressing how polluted the air is and what that means for health. Being able to forecast it a few days out, even imperfectly, gives people, institutions, and public health authorities something concrete to act on: whether to limit outdoor exposure, issue an advisory, or simply plan a school sports day around it. That matters more in a city like Karachi than in most places, since pollution here swings sharply with traffic patterns, weather, and season.

With that as the motivation, this project set out to build a machine learning pipeline that could:

- Collect historical and live AQI and weather data
- Turn that raw data into features useful for time-series forecasting
- Train, compare, and tune several regression models against each other
- Run the whole feature-and-training loop automatically, without manual intervention
- Explain individual predictions rather than treat the model as a black box
- Present forecasts and alerts through a dashboard anyone could open and understand

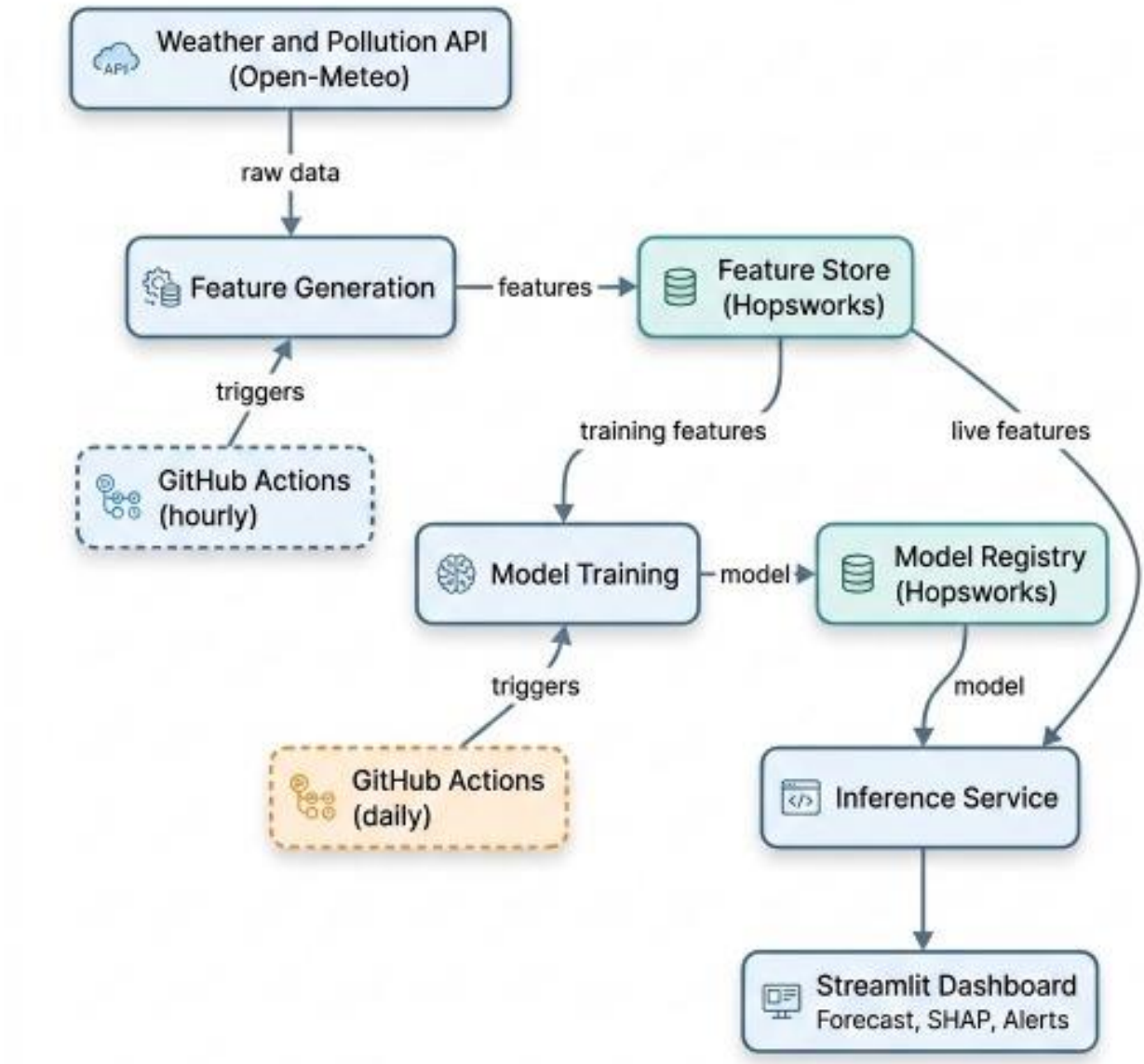
2. Project Objectives

The following objectives were established to guide the development of the AQI forecasting system:

- Ingest raw weather and pollutant data from the Open-Meteo API and engineer relevant features and target variables
- Persist engineered features in a centralized Feature Store
- Backfill sufficient historical feature-target pairs to support robust model training
- Train and evaluate multiple models (Linear Regression, Ridge, Random Forest, and Gradient Boosting) using RMSE, MAE, and R^2 as performance metrics
- Register the winning model in a Model Registry
- Automate the feature pipeline on an hourly basis and the training pipeline on a daily basis using CI/CD
- Serve predictions through a Streamlit-based web application
- Implement SHAP for model interpretability and feature importance explanation
- Integrate hazard alerts within the dashboard to notify users when forecasted AQI reaches dangerous levels

3. System Architecture

The system adopts a serverless architecture and decomposes functionality into distinct stages for feature management, model management, automation, and serving, rather than consolidating all operations within a single script.



4. Data Collection

Historical and live data for Karachi came from Open-Meteo's Air Quality and Weather APIs, queried against the following coordinates:

- Latitude: 24.8608
- Longitude: 67.0104
- Time zone: Asia/Karachi

Variables collected:

Category	Variables
Air Quality	PM2.5, PM10, Nitrogen Dioxide (NO_2), Sulphur Dioxide (SO_2), Ozone (O_3), US AQI
Weather	Temperature (2m), Relative Humidity (2m), Wind Speed (10m), Wind Direction (10m)

One choice that took some deliberation was which of Open-Meteo's two APIs to train on. The *archive* API is more accurate but lags roughly two days behind real time; the *forecast* API is current but noisier. Rather than assume one was obviously better, a controlled experiment was run, same 90-day window, same horizon, only the data source changed:

Model / Source	RMSE	MAE	R^2
Gradient Boosting (Archive API)	7.89	6.44	-0.34
Gradient Boosting (Forecast API)	9.87	7.93	-0.69

The archive-trained model came out roughly 20% lower on RMSE, which settled the question: training happens on the archive API despite the reporting lag, and that tradeoff is treated as a known, accepted design decision rather than a hidden compromise.

- **4.1 Backfilling Historical Data**

The project specification required a backfill mechanism to generate sufficient historical feature-target pairs for model training. However, a dedicated backfill script was deemed unnecessary in this implementation. The run_training_features() in feature_pipeline does that job. It pulls a full 365-day history, builds features for all three forecast horizons, and pushes them to the Hopsworks training feature group. We ran it once to populate features, which satisfies the backfill requirement, and it can be re-run anytime the training data needs refreshing Consequently, the backfilling requirement is satisfied by default.
This is separate from hourly automated job, which only fetches a small 14-day window to keep live dashboard predictions current.

5. Exploratory Data Analysis (EDA)

A quick EDA was run on a 7-day sample of hourly AQI and weather data for Karachi, and then the two sources were merged. There were some gaps in the data which were filled using forward-fill followed by back-fill.

Python

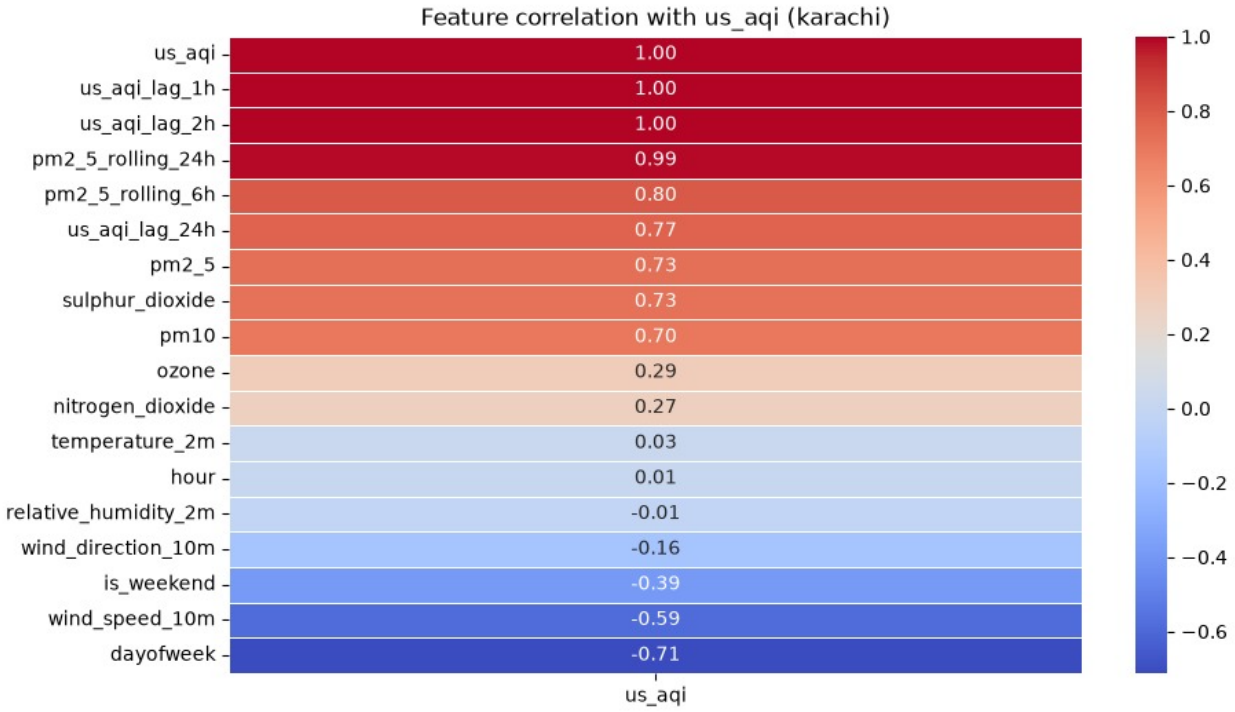
```
Total hourly rows:288, Total features:10
```

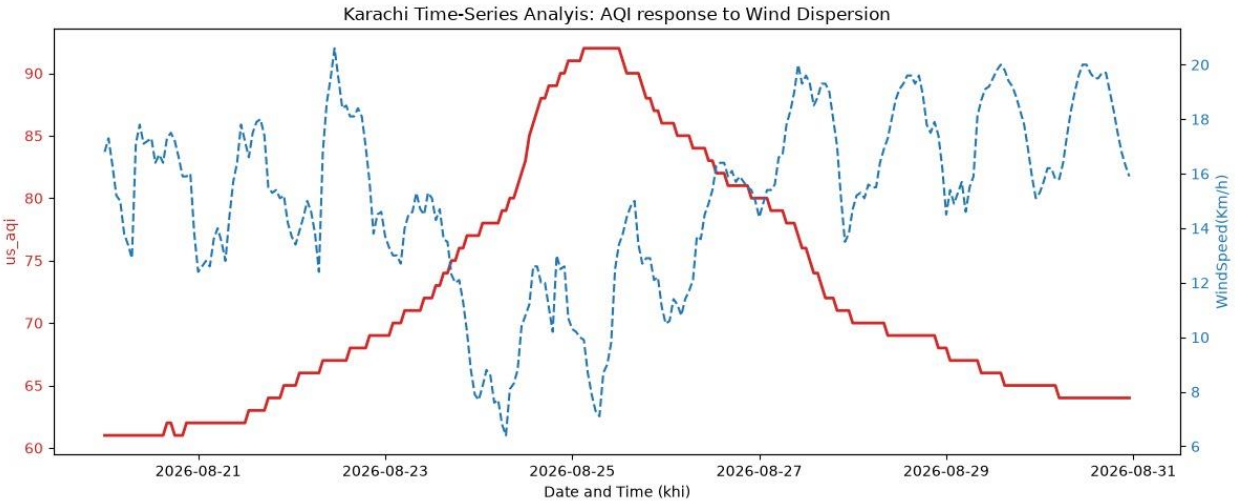
time	pm2_5	pm10	nitrogen_dioxide	sulphur_dioxide	ozone	us_aqi	temperature_2m	relative_humidity_2m	wind_speed_10m	wind_direction_10m
2026-08-19 00:00:00	14.9	28.9	8.5	4.6	38.0	66.0	27.1	81	15.7	257
2026-08-19 01:00:00	14.0	27.9	7.0	4.2	38.0	66.0	27.0	82	15.3	262
2026-08-19 02:00:00	13.2	26.1	5.8	3.9	39.0	66.0	26.8	83	15.3	261
2026-08-19 03:00:00	12.6	25.1	5.1	3.9	39.0	65.0	26.8	84	15.4	260
2026-08-19 04:00:00	12.0	23.5	4.8	4.0	39.0	65.0	26.7	85	15.7	260

```python
df_karachi.isnull().sum() #validating that there are no missing values
```

```
pm2_5                0
pm10                 0
nitrogen_dioxide     0
sulphur_dioxide      0
ozone                0
us_aqi               0
temperature_2m       0
relative_humidity_2m 0
wind_speed_10m       0
wind_direction_10m   0
dtype: int64
```

The correlation results were fairly telling. AQI's own recent history dominates everything else the 1h and 2h lags both correlate above 0.99 with current AQI, and so does the 24h rolling PM2.5 average. Among the raw pollutants, PM2.5, sulphur dioxide, and PM10 showed the strongest positive correlation with AQI. Wind speed had a strong negative correlation of -0.59, consistent with the plot below showing AQI dropping as wind picks up.


Feature correlation with us_aqi (karachi)
us_aqi 1.00
us_aqi_lag_1h 1.00
us_aqi_lag_2h 1.00
pm2_5_rolling_24h 0.99
pm2_5_rolling_6h 0.80
us_aqi_lag_24h 0.77
pm2_5 0.73
sulphur_dioxide 0.73
pm10 0.70
ozone 0.29
nitrogen_dioxide 0.27
temperature_2m 0.03
hour 0.01
relative_humidity_2m -0.01
wind_direction_10m -0.16
is_weekend -0.39
wind_speed_10m -0.59
dayofweek -0.71
us_aqi
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6


Karachi Time-Series Analyis: AQI response to Wind Dispersion
us_aqi
WindSpeed(Km/h)
90
85
80
75
70
65
60
20
18
16
14
12
10
8
6
2026-08-21
2026-08-23
2026-08-25
2026-08-27
2026-08-29
2026-08-31
Date and Time (khi)

6. Feature Engineering

Several engineered features were created to improve forecasting performance.

Time Features

Hour (raw + cyclical sin/cos encoding)
Day
Month
Day of Week
Weekend Indicator

Lag Features

us_aqi_lag_1h
us_aqi_lag_2h
us_aqi_lag_24h
us_aqi_lag_48h

Rolling Statistics

us_aqi_roll_mean_6h
us_aqi_roll_std_6h
us_aqi_roll_mean_24h
us_aqi_roll_std_24h

Rate-of-Change Features

us_aqi_change_rate_1h
us_aqi_change_rate_24h

Target Variable

The forecasting target was defined as:

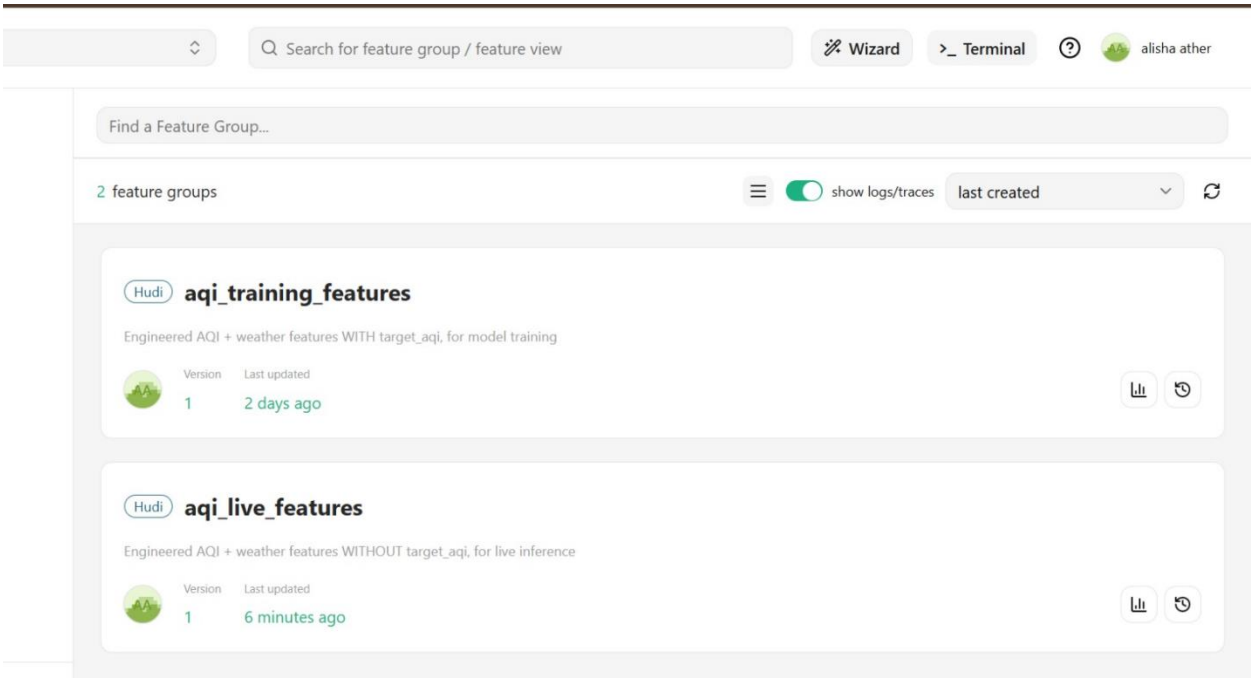
target_aqi = us_aqi.shift(-horizon)

where horizon is 24, 48, or 72 hours depending on which of the three models is being trained . This allows direct prediction at each forecast distance rather than a single fixed target.

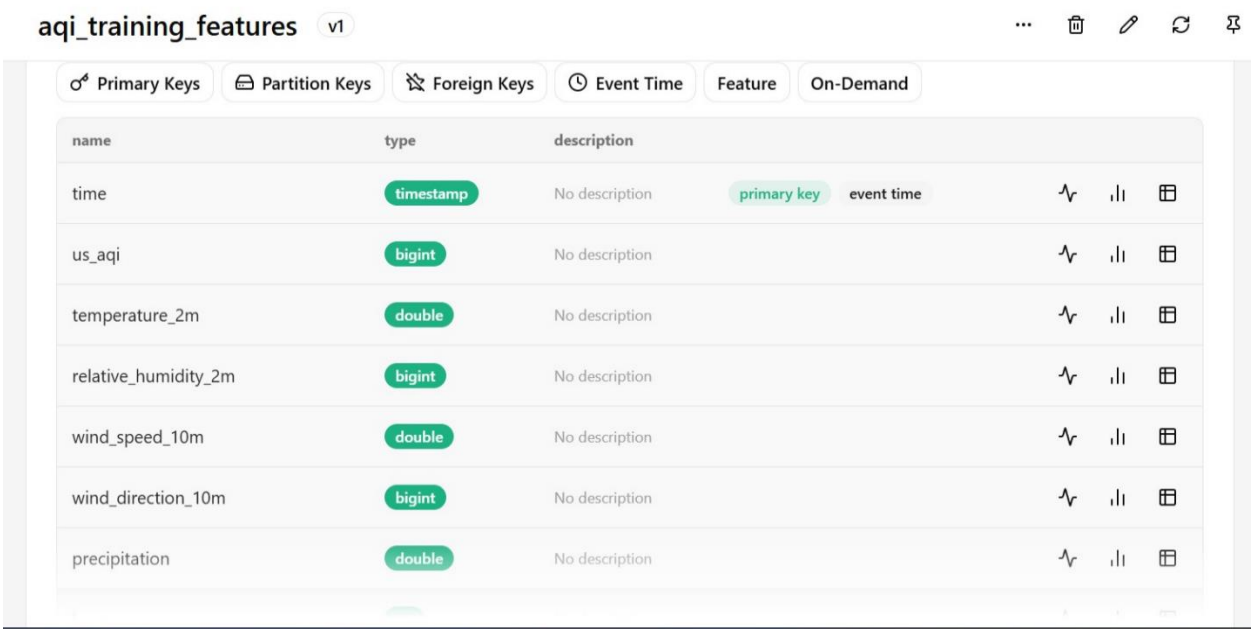
7. Feature Store Implementation

Hopsworks was chosen as the Feature Store.

Two feature groups are maintained: one holding historical data for training, and a separate one holding live data for inference. Keeping them apart meant the same feature logic didn't need to be duplicated between training and serving, the two stayed consistent with each other, and the system as a whole scaled and versioned cleanly. A dedicated script handles fetching the data, building features for all three forecast horizons, and pushing the results into the training feature group.



The training feature group also uses "horizon" as part of a composite primary key alongside time, since a single timestamp appears once for each of the three forecast horizons, with a different target value shifted accordingly for each.



8. Machine Learning Models

Five models were compared against one another for each forecast horizon:

- Persistence Baseline, which naively assumes AQI remains at its most recent known value.
- Linear Regression
- Ridge Regression
- Random Forest
- Gradient Boosting.

9. Model Evaluation

The comparison used time-series cross-validation rather than a standard k-fold split. This distinction is significant, as a random split can allow future information to leak into training folds when working with time-series data.

RMSE and MAE were used as the metrics that determined model selection, while R^2 was calculated and reported for completeness but was deliberately not used as the deciding criterion. The metric is inherently harsh in low-variance settings, and relying on it as the primary criterion would have produced a misleading assessment.

- **24-hour horizon**

Model	RMSE	MAE	R^2
Persistence Baseline	16.37	12.06	0.14
Linear Regression	19.21	15.59	-0.24
Ridge Regression	18.95	15.33	-0.21
Random Forest	18.12	13.92	-0.06
Gradient Boosting	15.46	11.81	0.24

- **48-hour horizon**

Model	RMSE	MAE	R^2
Persistence Baseline	20.15	15.22	-0.36
Linear Regression	38.99	35.12	-6.14
Ridge Regression	37.99	34.13	-5.70
Random Forest	23.34	18.69	-0.83
Gradient Boosting	18.77	14.58	-0.18

- **72-hour horizon**

Model	RMSE	MAE	R^2
Persistence Baseline	22.20	16.83	-0.70
Linear Regression	52.94	48.64	-13.40
Ridge Regression	51.33	47.09	-12.40
Random Forest	26.48	21.42	-1.36
Gradient Boosting	20.27	16.20	-0.38

Best Model

Gradient Boosting consistently outperformed the other models.

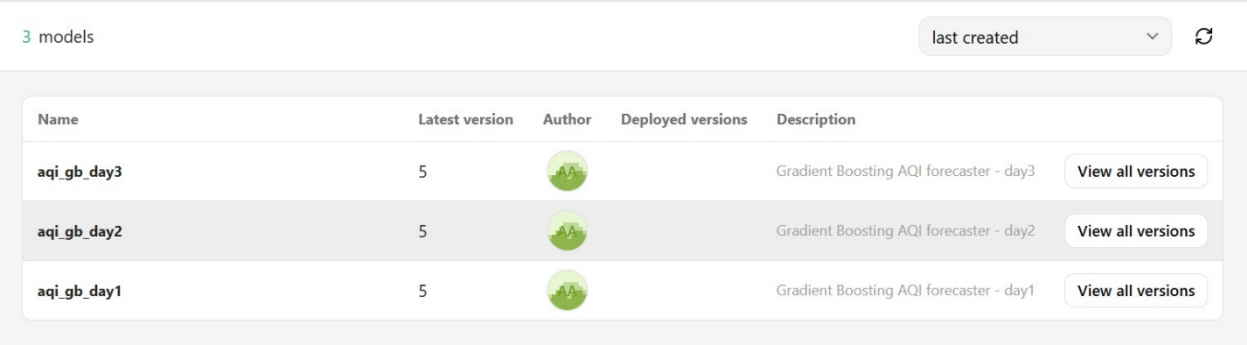
The tuned model improved on both RMSE and MAE relative to the baseline. The negative R^2 for both models is consistent with the low-variance explanation above, rather than a sign that either model is performing poorly in absolute terms.

10. Model Registry

The selected Gradient Boosting model is registered per horizon in the Hopsworks Model Registry, which manages versioning and reproducibility and provides a clean transition into deployment.

The training pipeline operates in two distinct modes:

1. The lightweight mode reloads data from Hopsworks and retrains the already-tuned Gradient Boosting models for all three horizons. This is the mode that runs automatically each day.
2. A separate comparison mode reruns the full five-model comparison used to generate the results discussed in Sections 8 and 9, but is intentionally excluded from the automated pipeline, since repeating a full model comparison daily would be unnecessarily slow once the best-performing model has already been established.



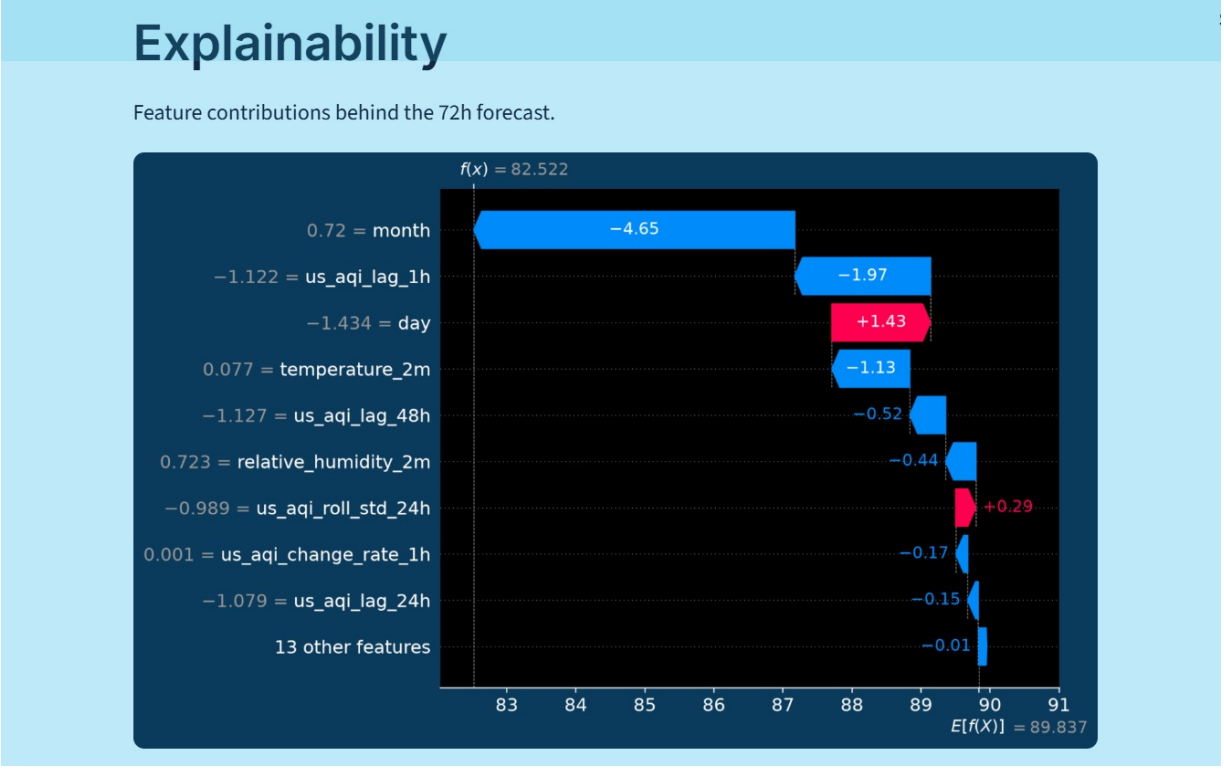
11. Forecasting & Inference System

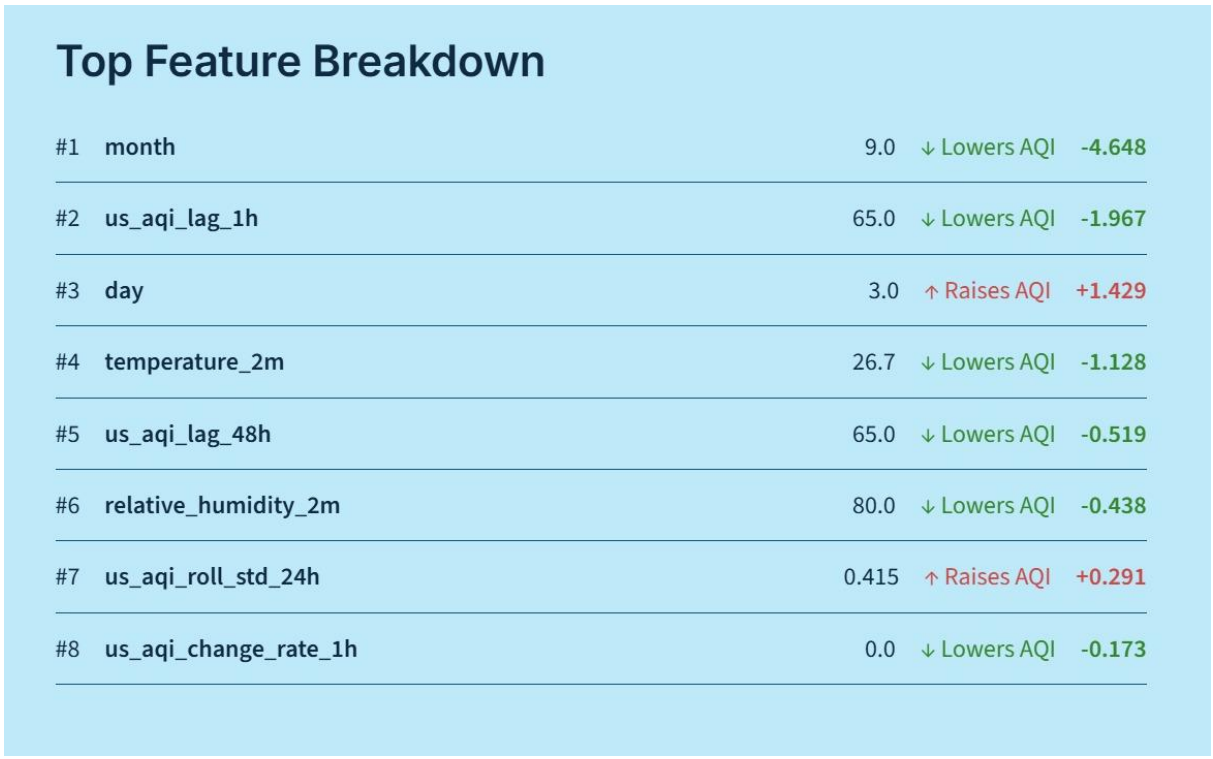
The inference service functions as the single point of access the dashboard relies on. It loads the most recent live features from the Feature Store, retrieves the appropriate horizon-specific model from the Model Registry, and returns the direct 24-hour, 48-hour, and 72-hour forecast.

Because the forecasting approach is direct rather than recursive, each horizon is predicted independently from the current feature state, rather than being derived from an earlier prediction. This prevents forecast error from compounding across the three-day window.

12. Explainability Using SHAP

The dashboard generates a live SHAP waterfall plot for the current 72-hour prediction, showing which features pushed that specific forecast up or down and by how much. This gives users a concrete, per-prediction explanation rather than a generic feature-importance ranking.





Raw pollutant concentrations (PM2.5, PM10, NO_2, SO_2, ozone) never appear in the SHAP output, since they were deliberately excluded from the feature set before training to avoid redundancy with AQI's own lag and rolling statistics, which the EDA showed carried a much stronger signal. Because SHAP can only attribute importance to features a model actually saw during training, these pollutants have no representation in the explanation, even though they causally underlie the AQI value itself.

13. AQI Alert System

The dashboard flags any forecast horizon where predicted AQI exceeds 150, surfacing a health advisory for that specific horizon rather than a single blanket warning. If none of the three horizons cross that threshold, a clear "no hazardous levels expected" message is shown instead.

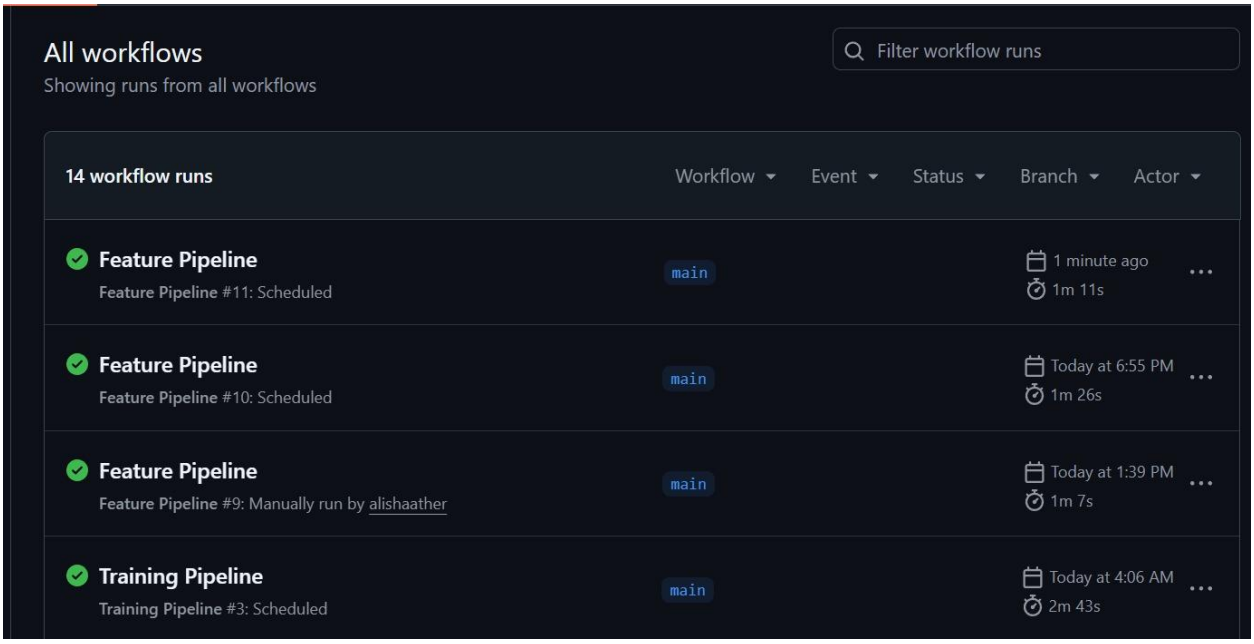


14. CI/CD Automation

Both pipelines run through GitHub Actions rather than any manual step.

Feature Pipeline: Runs hourly, fetching the latest weather and pollutant data, engineering features, and pushing the results to the live feature group in Hopsworks. This keeps the data behind the dashboard's real-time predictions current without any manual intervention.

Training Pipeline: Runs daily at midnight PKT, retraining and registering the Gradient Boosting models for all three forecast horizons, then committing the updated model files back to the repository.



15. Dashboard Implementation

The dashboard is built with Streamlit. It shows the current live AQI for Karachi with its severity category, the three-day forecast broken into separate cards for each horizon, a 7-day AQI trend chart, a live SHAP plot explaining the 72-hour prediction, and a hazard alert whenever any forecasted horizon crosses an AQI of 150.

The dashboard's current AQI and trend chart lag roughly two days behind real time. This is because AQI and weather data are merged on a shared timestamp, and the weather source is only available up to two days prior so the merged dataset caps at that same point, even though AQI readings alone are more current. (fig 2)

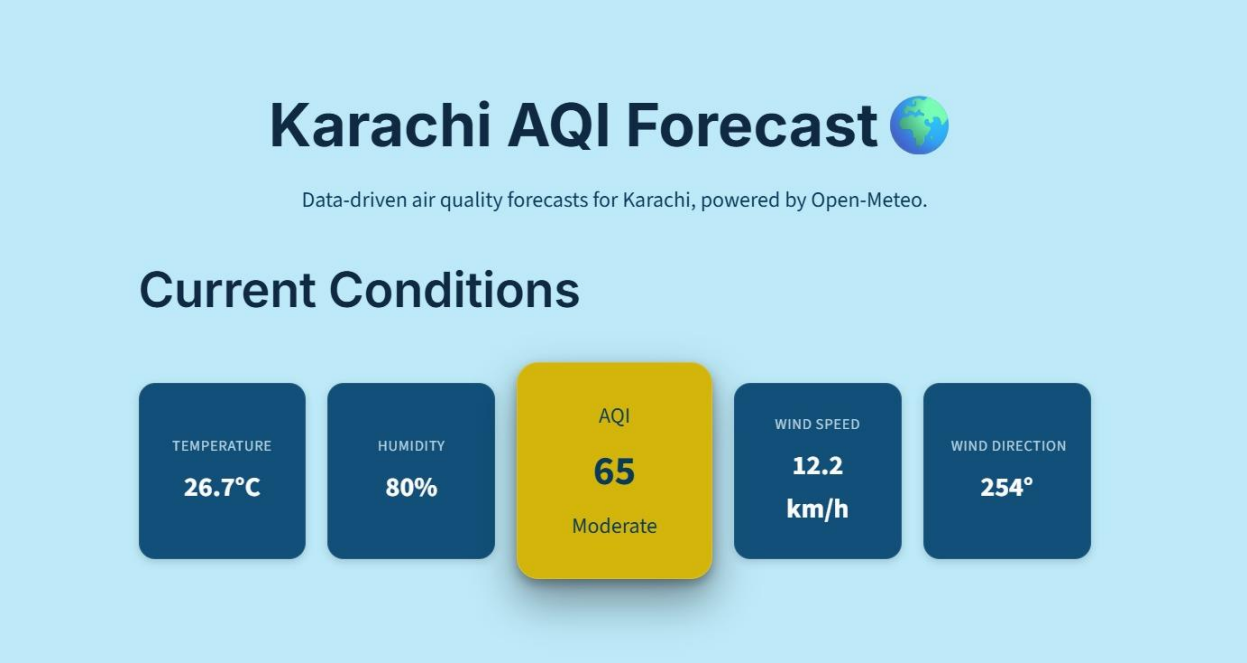

Karachi AQI Forecast
Data-driven air quality forecasts for Karachi, powered by Open-Meteo.
Current Conditions
TEMPERATURE
26.7°C
HUMIDITY
80%
AQI
65
Moderate
WIND SPEED
12.2
km/h
WIND DIRECTION
254°

fig 1

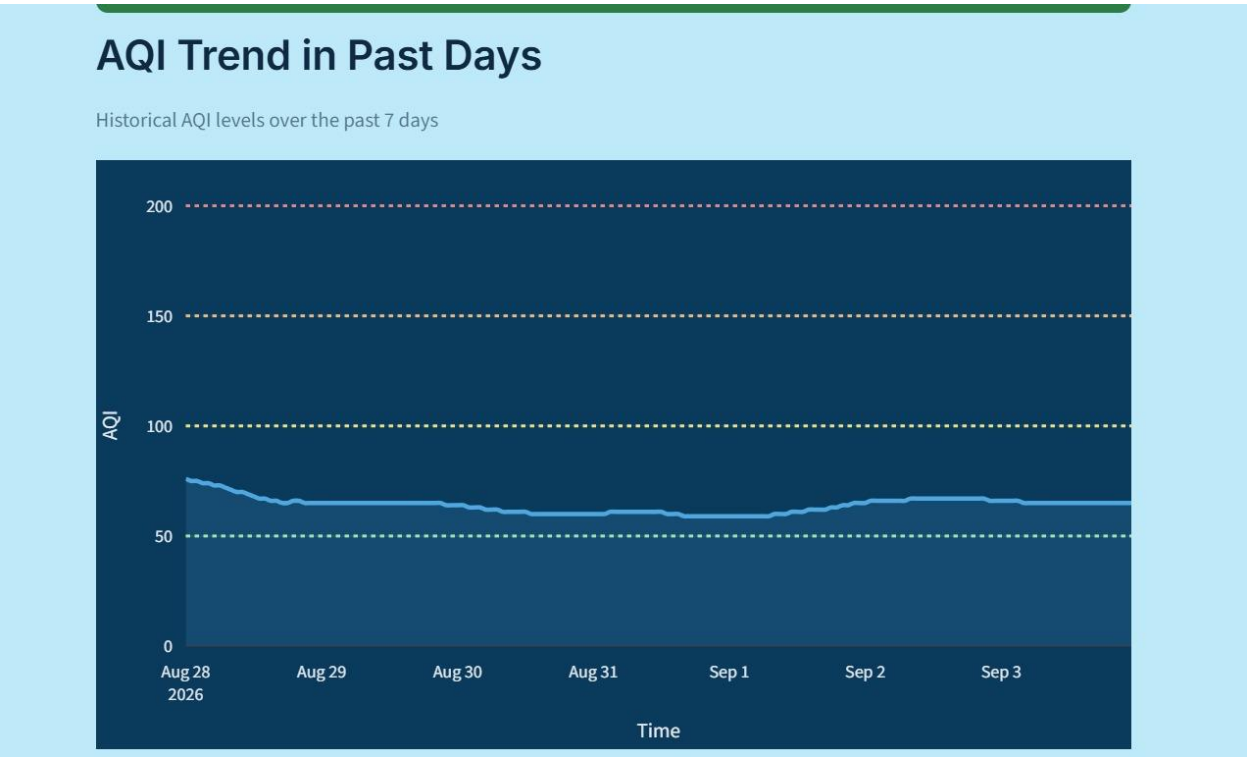

AQI Trend in Past Days
Historical AQI levels over the past 7 days
200
150
100
50
0
AQI
Aug 28
2026
Aug 29
Aug 30
Aug 31
Sep 1
Sep 2
Sep 3
Time

fig 2

16. Key Challenges & Resolutions

Several issues arose during development that warrant detailed documentation rather than a brief mention.

The first was a clear case of data leakage. The initial model reported an R^2 of 0.99, a result that should have prompted scrutiny rather than confidence. Investigation revealed that the previous hour's AQI value had been used as the training target instead of a properly shifted future value. The issue was resolved by explicitly shifting the target forward by the forecast horizon.

The second issue was subtler. Training features for different forecast horizons were overwriting one another within the Feature Store, because the primary key used was time alone, with no means of distinguishing between horizons. This was resolved by implementing a composite primary key consisting of both time and horizon.

Establishing a working Hopsworks integration required considerable effort in its own right. This included resolving project creation errors, addressing Python version conflicts caused by the removal of a required module in newer Python releases, installing the C++ build tools needed to compile a required dependency.

Hopsworks does not guarantee chronological row order on read. Since time-series cross-validation assumes row order reflects actual time order, this silently corrupted validation folds, mixing past and future data without raising any error. The fix was to explicitly sort by timestamp immediately after reading data back from the feature store, before running cross-validation.

Finally, the CI/CD pipeline itself failed on several occasions during early development, due to missing dependencies, insufficient permissions, and the data type mismatch described above. Each of these issues was resolved as detailed in Section 14.

17. Technologies Used

Component	Technology
Programming Language	Python
Data Source	Open-Meteo
Data Processing	Pandas, NumPy
Feature Store	Hopsworks
Model Registry	Hopsworks
Machine Learning Models	Linear Regression, Ridge Regression, Random Forest, Gradient Boosting
Explainability	SHAP
Dashboard	Streamlit
CI/CD Automation	GitHub Actions
Version Control	Git & GitHub

18. Future Improvements

There's a reasonable list of things this project could still take further. Deep learning models, an LSTM or similar, built in PyTorch, can be implemented. Extending the system beyond Karachi to other cities is a natural next step, as is adding real-time push notifications rather than relying on someone checking the dashboard. And longer-range weather forecasts could eventually be folded into the feature set for a more weather-aware ensemble approach.

19. Conclusion

What this project set out to build, and largely did, is a working AQI forecasting system for Karachi that doesn't just produce a number but does so through a defensible, automated pipeline, direct multi-horizon prediction, time-aware training, and a genuine MLOps setup built on Hopsworks and GitHub Actions rather than a one-off script.

The project satisfies all major requirements of the internship challenge while providing a practical and extensible solution for air quality forecasting.

GitHub Link: https://github.com/alishaather/aqi_prediction

Streamlit App Link: https://aqiprediction-karachi.streamlit.app/